

# Лабораторная работа № 5 по курсу дискретного анализа: суффиксные деревья

Выполнил студент группы М80-308Б-22 МАИ *Цирулев Николай*.

## Условие

Вариант №2: Поиск с использованием суффиксного массива

Найти в заранее известном тексте поступающие на вход образцы с использованием суффиксного массива.

Формат ввода

Текст располагается на первой строке, затем, до конца файла, следуют строки с образцами.

Формат вывода

Для каждого образца, найденного в тексте, нужно распечатать строчку, начинающуюся с последовательного номера этого образца и двоеточия, за которым, через запятую, нужно перечислить номера позиций, где встречается образец в порядке возрастания.

## Метод решения

Алгоритм строит массив суффиксов строки  $s$  (индексы её суффиксов в лексикографическом порядке) за  $O(n \log n)$ , уточняя порядок суффиксов с помощью поразрядной сортировки и классов эквивалентности. Сначала суффиксы сортируются по первому символу с использованием сортировки подсчетом. Затем каждому суффиксу назначается класс эквивалентности: одинаковые символы получают один класс, разные — разные.

После этого порядок уточняется на каждом шаге, увеличивая длину учитываемого префикса вдвое. Для каждого суффикса рассматривается пара: левая часть (первые  $2^h$  символов) и правая часть (следующие  $2^h$  символов). Сортировка выполняется по текущим классам эквивалентности, также обновляются классы для следующей итерации.

Процесс продолжается, пока длина префикса не превысит длину строки. В результате массив суффиксов  $p$  содержит индексы всех суффиксов строки в лексикографическом порядке. Алгоритм эффективен благодаря разделению задачи на последовательные этапы с использованием сортировки подсчетом.

Поиск паттерна осуществляется бинарным поиском по суффиксному массиву и тривиальным сравнением текущего суффикса и подстроки.

## Описание программы

```
#include <iostream>
#include <string>
#include <vector>
```

```

#include <algorithm>

using namespace std;

std::vector<int> BuildSuffixArray(const std::string& s) {
    const int n = s.size();
    if (n == 0) {
        return {};
    }

    const int alphabet = *std::max_element(s.begin(), s.end()) + 1;
    std::vector<int> p(n), cnt(alphabet, 0), c(n), pn(n), cn(n);

    for (int i = 0; i < n; ++i)
        ++cnt[s[i]];
    for (int i = 1; i < alphabet; ++i)
        cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; --i)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; ++i) {
        if (s[p[i]] != s[p[i - 1]]) ++classes;
        c[p[i]] = classes - 1;
    }

    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; ++i) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0) pn[i] += n;
        }
        cnt.assign(classes, 0);
        for (int i = 0; i < n; ++i)
            ++cnt[c[pn[i]]];
        for (int i = 1; i < classes; ++i)
            cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; --i)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; ++i) {
            int mid1 = (p[i] + (1 << h)) % n, mid2 = (p[i - 1] + (1 << h))

```

```

        if (c[p[i]] != c[p[i - 1]] || c[mid1] != c[mid2])
            ++classes;
        cn[p[i]] = classes - 1;
    }
    std::swap(c, cn);
}
return p;
}

```

```

vector<int> SlowSearch(int l, int r, const string& str, const vector<int>&
vector<int> res;
if (l > r) {
    return res;
}
int m = (l + r) / 2;
int cmp = str.compare(p[m], pat.size(), pat);
if (cmp == 0) {
    res.push_back(p[m]);
    vector<int> lres = SlowSearch(l, m - 1, str, p, pat);
    res.insert(res.end(), lres.begin(), lres.end());
    vector<int> rres = SlowSearch(m + 1, r, str, p, pat);
    res.insert(res.end(), rres.begin(), rres.end());
    return res;
} else if (cmp < 0) {
    return SlowSearch(m + 1, r, str, p, pat);
} else {
    return SlowSearch(l, m - 1, str, p, pat);
}
return res;
}

```

```

int main() {
    string str, pat;
    getline(cin, str);
    str += '\0';
    vector<int> p = BuildSuffixArray(str);
    unsigned int i = 0;
    while (cin >> pat) {
        i++;
        vector<int> res = SlowSearch(0, str.size() - 1, str, p, pat);
        std::sort(res.begin(), res.end());
    }
}

```

```

        if (res.size() != 0) {
            std::cout << i << ":\n";
        }
        for (size_t j = 0; j < res.size(); j++) {
            cout << res[j] + 1 << (j == res.size() - 1 ? "\n" : ",\n");
        }
    }
}

```

## Дневник отладки

После отправки решения в чекер не было обнаружено ошибок.

## Тест производительности

Асимптотика создания суффиксного массива -  $O(|T| \log(|T|))$ , асимптотика поиска -  $O(|P| \log(|T|))$ , где  $|P|$  - длина паттерна,  $|T|$  - длина текста. Для лучшей наглядности приведём таблицу, в которой время работы алгоритма сопоставляется с вводимыми данными.

| $ T $ (Длина текста) | $ P $ (Длина паттерна) | Время создания суффиксного массива, мс | Время работы алгоритма поиска, мс |
|----------------------|------------------------|----------------------------------------|-----------------------------------|
| 1000                 | 38                     | 1055                                   | 21                                |
| 10000                | 491                    | 13217                                  | 37                                |
| 100000               | 847                    | 191180                                 | 218                               |
| 1000000              | 248                    | 6587616                                | 6388                              |
| 10000000             | 127                    | 110419063                              | 95625                             |

Ниже приведена программа benchmark.cpp, использовавшаяся для определения времени работы алгоритма:

```

#include <random>
#include <chrono>
#include "main"

int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution short_dist(1, 100);
    std::uniform_int_distribution<char> char_dist(0, 25);
    for (int k = 10; k < 1e10; k *= 10) {
        std::string str;
        std::uniform_int_distribution int_dist(1, k/5);
        std::vector<std::string> patterns;
    }
}

```

```

int count = 0;
for (int i = 0; i < k; ++i) {

    std::string p;
    int n = int_dist(gen);
    for (int j = 0; j < n; ++j) {
        p += static_cast<char>('a' + char_dist(gen));
    }
    patterns.push_back(p);
    int flag = short_dist(gen);
    if (flag == 10) {
        str += p;
        i += n;
        count++;
    }
    if (i >= k) {
        break;
    }
    str += static_cast<char>('a' + char_dist(gen));
}

std::cout << k << "_&_" << count << "_&_";

str += '\0';

auto start1 = std::chrono::high_resolution_clock::now();

vector<int> p = BuildSuffixArray(str);

auto finish1 = std::chrono::high_resolution_clock::now();
auto duration1 = std::chrono::duration_cast<std::chrono::microsecond>(finish1 - start1);

auto start2 = std::chrono::high_resolution_clock::now();
unsigned int i = 0;
for(int l = 0; l < count; l++) {
    i++;
    vector<int> res = SlowSearch(0, str.size() - 1, str, p, patterns[l]);
    std::sort(res.begin(), res.end());
    //if (res.size() != 0) {
    //    std::cout << i << ": ";
    //}
    //for (size_t j = 0; j < res.size(); j++) {

```

```

        //      cout << res[j] + 1 << (j == res.size() - 1 ? "\n" : ", ");
        //}
    }
    auto finish2 = std::chrono::high_resolution_clock::now();
    auto duration2 = std::chrono::duration_cast<std::chrono::microseconds>(finish2 - finish1);
    std::cout << duration1.count() << "_&" << duration2.count() << "\n";
    std::cout << '\n';
}
}

```

## Выводы

В ходе выполнения данной работы были подробно изучены суффиксные массивы, реализован алгоритм построения суффиксного массива без построения суффиксного дерева, а также наивный алгоритм поиска паттерна по тексту.

Тесты показали, что наивный алгоритм поиска не столь эффективен на больших объемах, существует более быстрое решение, использующее longest common prefix. Реализованный же алгоритм полезен лишь в учебных целях.